

Step1: Create a folder and create two files app.py and requirements.txt

app.py

```
from flask import Flask, request, jsonify
import sqlite3
import os

app = Flask(__name__)

# SQLite database
DATABASE = os.path.join(os.path.dirname(__file__), "students.db")

def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn

# CREATE - Add Student
@app.route('/students', methods=['POST'])
def add_student():
    data = request.get_json()
    name = data['name']
    email = data['email']
    course = data['course']
    conn = get_db_connection()
    conn.execute(
        "INSERT INTO students (name, email, course) VALUES (?, ?, ?)",
        (name, email, course)
    )
    conn.commit()
    conn.close()
    return jsonify({
        "message": "Student added successfully"
    }), 201
```

```

# READ - Get All Students
@app.route('/students', methods=['GET'])
def get_students():
    conn = get_db_connection()
    students = conn.execute(
        "SELECT * FROM students"
    ).fetchall()
    conn.close()
    result = []
    for student in students:
        result.append({
            "id": student["id"],
            "name": student["name"],
            "email": student["email"],
            "course": student["course"]
        })
    return jsonify(result), 200

# UPDATE - Update Student
@app.route('/students/<int:id>', methods=['PUT'])
def update_student(id):
    data = request.get_json()
    name = data['name']
    email = data['email']
    course = data['course']
    conn = get_db_connection()
    conn.execute(
        """UPDATE students
        SET name = ?, email = ?, course = ?
        WHERE id = ?""",
        (name, email, course, id)
    )

```

```
conn.commit()
conn.close()
return jsonify({
    "message": "Student updated successfully"
}), 200

# DELETE - Delete Student
@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    conn = get_db_connection()
    conn.execute(
        "DELETE FROM students WHERE id = ?",
        (id,)
    )
    conn.commit()
    conn.close()
    return jsonify({
        "message": "Student deleted successfully"
    }), 200

if __name__ == '__main__':
    app.run(debug=True)
```

requirements.txt

```
Flask
Flask-MySQLdb
mysqlclient
```

Step2: add all the files to ur github

git init

git add .

git commit -m "first commit"

go to ur github and create a new repository & copy its url and in vs code terminal

git remote add origin <url>

git branch -M main

git push -u origin main

Step 3: Go to python anywhere and create free account

- Dashboard open bash console and type
git clone <github url>
ls
cd <folder/repositoryname>
ls o/p -shows all files in repo
- Create virtual environment
mkvirtualenv --python=/usr/bin/python3.13 studentapi
- Install requirements.txt
pip install -r requirements.txt

Step 4: create database

type python in bash

and code

```
import sqlite3

conn = sqlite3.connect("students.db")
conn.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    course TEXT NOT NULL
)
""")
conn.commit()
conn.close()

print("Database created successfully")
```

do ls and shld show students.db

Step 5: python anywhere -> dashboard-> web-> add a new web app

choose manual configuration->choose python version and click next web page will be created

configure wsgi.py

remove the old code

and new code and save

```
import sys

path = '/home/lavanya/pythonanywhere-demo'

if path not in sys.path:
    sys.path.insert(0, path)

from app import app as application
```

then in virtualenv enter the url of environment created from bash console

```
13:29 ~/pythonanywhere-demo (main) $ mkvirtualenv --python=/usr/bin/python3.13 studentapi
created virtual environment CPython3.13.1.final.0-64 in 11961ms
creator CPython3Posix(dest=/home/lavanya/.virtualenvs/studentapi, clear=False, no_vcs_ignore=False, global=False)
```

click on button reload and after reload open the link (username.pythonanywhere.com)

-----*****-----

Note: mysql doesnt work for free version we shall be using sqlite 3 instead of avien cloud database



Bash console 48014575

```
13:23 - $ git clone https://github.com/LavanyaR007/pythonanywhere-demo.git
Cloning into 'pythonanywhere-demo'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
13:28 - $ ls
bash: ls: command not found
13:29 - $ ls
README.txt  pythonanywhere-demo
13:29 - $ cd pythonanywhere-demo
13:29 ~/pythonanywhere-demo (main)$ ls
app.py  requirements.txt
13:29 ~/pythonanywhere-demo (main)$ mkvirtualenv --python=/usr/bin/python3.13 studentapi
created virtual environment CPython3.13.1.final.0-64 in 11961ms
creator CPython3Posix(dest=/home/lavanya/.virtualenvs/studentapi, clear=False, no_vcs_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, via=copy, app_data_dir=/home/lavanya/.local/share/virtualenv)
added seed packages: pip==24.3.1
activators BashActivator,CSShellActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator
virtualenvwrapper.user_scripts creating /home/lavanya/.virtualenvs/studentapi/bin/predeactivate
virtualenvwrapper.user_scripts creating /home/lavanya/.virtualenvs/studentapi/bin/postdeactivate
virtualenvwrapper.user_scripts creating /home/lavanya/.virtualenvs/studentapi/bin/preactivate
virtualenvwrapper.user_scripts creating /home/lavanya/.virtualenvs/studentapi/bin/postactivate
virtualenvwrapper.user_scripts creating /home/lavanya/.virtualenvs/studentapi/bin/get_env_details
(studentapi) 13:31 ~/pythonanywhere-demo (main)$ pip install -r requirements.txt
ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements.txt'
(studentapi) 13:32 ~/pythonanywhere-demo (main)$ pip install -r requirements.txt
Looking in links: /usr/share/pip-wheels
Collecting Flask (from -r requirements.txt (line 1))
  Downloading Flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting Flask-MySQLdb (from -r requirements.txt (line 2))
  Downloading Flask-MySQLdb-2.0.0-py3-none-any.whl.metadata (3.0 kB)
Collecting mysqlclient (from -r requirements.txt (line 3))
  Downloading mysqlclient-2.2.8.tar.gz (92 kB)
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Collecting blinker>=1.9.0 (from Flask->-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask->-r requirements.txt (line 1))
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask->-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask->-r requirements.txt (line 1))
```



Bash console 48014575

Share with others

```
Successfully built mysqlclient
Installing collected packages: mysqlclient, markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask, flask-MySQLdb
Successfully installed Flask-3.1.3 Flask-MySQLdb-2.0.0 blinker-1.9.0 click-8.5.0 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 mysqlclient-2.2.8 werkzeug-3.1.8
(studentapi) 13:33 ~/pythonanywhere-demo (main)$ workon studentapi
(studentapi) 14:03 ~/pythonanywhere-demo (main)$ python
Python 3.13.1 (main, Jan 16 2025, 13:50:41) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import MySQLdb
>>> MySQLdb.connect(
...     host="mysql-32c04060-lavusarj18-daf7.j.aivencloud.com",
...     user="avnadmin",
...     passwd="AvNs_Cj6j06Xh14et6z2w2Pt",
...     db="defaultdb",
...     port=11660
... )
Traceback (most recent call last):
  File "/home/lavanya/.virtualenvs/studentapi/lib/python3.13/site-packages/MySQLdb/__init__.py", line 111, in connect
    return Connection(*args, **kwargs)
  File "/home/lavanya/.virtualenvs/studentapi/lib/python3.13/site-packages/MySQLdb/connection.py", line 206, in __init__
    self._connect()
  File "/home/lavanya/.virtualenvs/studentapi/lib/python3.13/site-packages/MySQLdb/connection.py", line 214, in _connect
    self._db = self._db.encode('utf-8')
MySQLdb.OperationsError: (2002, "Can't connect to MySQL server on 'mysql-32c04060-lavusarj18-daf7.j.aivencloud.com:11660' (111)")
>>> nc -vz mysql-32c04060-lavusarj18-daf7.j.aivencloud.com 11660
nc -vz mysql-32c04060-lavusarj18-daf7.j.aivencloud.com 11660
>>> telnet mysql-32c04060-lavusarj18-daf7.j.aivencloud.com 11660
File "mysqlclient/_mysql.py", line 1
```

```

>>> exit()
(studentapi) 14:08 ~/pythonanywhere-demo (main)$ nc -vz mysql-32c04060-lavusarj18-daf7.j.aivencloud.com 11660
nc: connect to mysql-32c04060-lavusarj18-daf7.j.aivencloud.com (139.59.38.145) port 11660 (tcp) failed: Connection refused
(studentapi) 14:08 ~/pythonanywhere-demo (main)$ mysql -h mysql-32c04060-lavusarj18-daf7.j.aivencloud.com -P 11660 -u avnadmin -p
Enter password:
ERROR 2003 (HY000): Can't connect to MySQL server on 'mysql-32c04060-lavusarj18-daf7.j.aivencloud.com:11660' (111)
(studentapi) 14:11 ~/pythonanywhere-demo (main)$ git pull origin main
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 11 (delta 4), reused 8 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (11/11), 2.24 KiB | 9.00 KiB/s, done.
From https://github.com/LavanyaR007/pythonanywhere-demo
  * branch      main      -> FETCH_HEAD
  * 5ea4735..cc8ellf  main    -> origin/main
Updating 5ea4735..cc8ellf
Fast-forward
 app.py      | 100 +-----
 students.db | 6 +-----
 2 files changed, 67 insertions(+), 39 deletions(-)
 create mode 100644 students.db
(studentapi) 14:29 ~/pythonanywhere-demo (main)$ cat app.py
from flask import Flask, request, jsonify
import sqlite3
import os

app = Flask(__name__)

# SQLite database
DATABASE = os.path.join(os.path.dirname(__file__), "students.db")

# Function to get database connection
def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn

# CREATE - Add Student
@app.route('/students', methods=['POST'])
def add_student():
    data = request.get_json()
    name = data['name']

```

```

(studentapi) 14:32 ~/pythonanywhere-demo (main)$ python
Python 3.13.1 (main, Jan 16 2025, 13:50:41) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import sqlite3

...
... conn = sqlite3.connect("students.db")
...
... conn.execute("""
...     CREATE TABLE IF NOT EXISTS students (
...         id INTEGER PRIMARY KEY AUTOINCREMENT,
...         name TEXT NOT NULL,
...         email TEXT NOT NULL,
...         course TEXT NOT NULL
...     )
... """)
...
... conn.commit()
... conn.close()
...
... print("Database created successfully")
...
KeyboardInterrupt
>>> import sqlite3
>>> conn = sqlite3.connect("students.db")
>>> conn.execute("""
...     CREATE TABLE IF NOT EXISTS students (
...         id INTEGER PRIMARY KEY AUTOINCREMENT,
...         name TEXT NOT NULL,
...         email TEXT NOT NULL,
...         course TEXT NOT NULL
...     )
... """)
<sqlite3.Cursor object at 0x7aa8bd343940>
>>> conn.commit()
... conn.close()
...
KeyboardInterrupt
>>> conn.commit()
>>> conn.close()
>>> print("Database created successfully")
Database created successfully
>>> exit()

```